30、spi-nand image制作

全志V3S嵌入式驱动开发（spi-nand image制作）

原创

嵌入式-老费 已于 2023-08-29 15:19:57 修改

阅读量1.2k 收藏 2

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 ›

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

摘要 本文详细介绍了如何在全志V3S平台上进行SPI-NAND驱动开发，包括准备xfel工具、下载编译uboot、修改u-boot-spl.bin文件、烧录splt.bin、修改kernel文件、编译烧录根文件系统，并进行了整体测试。通过这些步骤，成功实现了从SPI-NAND启动并验证了系统的可行性。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

上一篇文章，我们说到了spi-nor image的制作和输入。相比较spi-nor，spi-nand虽然在稳定性上面差一点，但是价格上面有很大的优势。举例来说，一般32M的spi-nor大约在6-7元左右，但是128M的spi-nand只需要3-4元，因此对客户来说，哪怕实际产品中有一些简单的视频和音频文件，只要使用的是spi-nand，实际使用中也没有多大的关系。

```
[    1.661317] ubi0: attaching mtd3
[    2.701343] ubi0: scanning is finished
[    2.719648] ubi0: attached mtd3 (name "rootfs", size 121 MiB)
[    2.725531] ubi0: PEB size: 131072 bytes (128 KiB), LEB size: 126976 bytes
[    2.732399] ubi0: min./max. I/O unit sizes: 2048/2048, sub-page size 2048
[    2.739219] ubi0: VID header offset: 2048 (aligned 2048), data offset: 4096
[    2.746187] ubi0: good PEBs: 974, bad PEBs: 1, corrupted PEBs: 0
[    2.752186] ubi0: user volume: 1, internal volumes: 1, max. volumes count: 128
[    2.759422] ubi0: max/mean erase counter: 2/0, WL threshold: 4096, image sequence number: 689011036
[    2.768465] ubi0: available PEBs: 0, total reserved PEBs: 974, PEBs reserved for bad PEB handling: 19
[    2.777766] sun6i-rtc 1c20400.rtc: setting system clock to 1970-01-01T00:00:07 UTC (7)
[    2.786002] vcc3v0: disabling
[    2.788980] vcc3v3: disabling
[    2.791977] vcc5v0: disabling
[    2.794998] ALSA device list:
[    2.797965]   No soundcards found.
[    2.801923] ubi0: background thread "ubi_bgt0d" started, PID 92
[    2.810158] UBIFS (ubi0:0): Mounting in unauthenticated mode
[    2.818439] UBIFS (ubi0:0): background thread "ubifs_bgt0_0" started, PID 93
[    2.891085] UBIFS (ubi0:0): recovery needed
[    3.079307] UBIFS (ubi0:0): recovery completed
[    3.083965] UBIFS (ubi0:0): UBIFS: mounted UBI device 0, volume 0, name "rootfs"
[    3.091390] UBIFS (ubi0:0): LEB size: 126976 bytes (124 KiB), min./max. I/O unit sizes: 2048 bytes/2048 bytes
[    3.101324] UBIFS (ubi0:0): FS size: 91549696 bytes (87 MiB, 721 LEBs), journal size 9023488 bytes (8 MiB, 72 LEBs)
[    3.111775] UBIFS (ubi0:0): reserved for root: 0 bytes (0 KiB)
[    3.117618] UBIFS (ubi0:0): media format: w4/r0 (latest is w5/r0), UUID 9C65E13B-34CA-4ECA-90F2-378C0F4EE003, small LPT model
[    3.131078] VFS: Mounted root (ubifs filesystem) on device 0:13.
[    3.141097] devtmpfs: mounted
[    3.145411] Freeing unused kernel memory: 1024K
[    3.150129] Run /sbin/init as init process
Starting logging: OK
Initializing random number generator... done.
Starting network: OK

Welcome to Buildroot
buildroot login:
```

CSDN @嵌入式-老费

前面我们谈过了kernel中spi-nand驱动怎么处理，也涉及到spi-nand image是怎么用usb otg进行烧入，但是这中间有一个核心的问题没有涉及到，那就是spi-nand image应该如何在制作？毕竟之前的image都是电商厂家直接给出来的。

要做好spi-nand image的制作，有三个重要的问题会涉及到，第一，最初的uboot应该怎么被写入；第二，怎么处理uboot spl写入的问题；第三，怎么制作根文件系统的问题。

1、准备xfel工具

之前谈spi-nor image准备的时候，当时用了sunxi-fel这个工具。而如果需要烧入spi-nand image，这个时候就需要另外一个工具，即xfel，它的下载地址如下，

```
https://github.com/xboot/xfel
```

下载的时候直接选master即可。等下载结束后，导入到ubuntu虚拟机，解压。因为里面已经包含了makefile文件，所以直接输入一个make命令就好，

```
make
```

此时如果ls一下，就可以看到生成的xfel这个工具，利用它，就可以完成最初uboot的烧入工作。 **当然，如果要使用这个命令，也同样要求v3s必须进入usb otg状态，这和sunxi-fel是一样的。** 这里，我们可以先简单熟悉下xfel的命令，

```
sudo ./xfel version
sudo ./xfel spinand
sudo ./xfel spinand erase 0x0 0x100000
sudo ./xfel spinand write 0x0 split.bin
sudo ./xfel spinand read 0x0 0x1000 4k.bin
```

2、下载和编译uboot代码

早期的uboot是不支持spi-nand启动的。因此，要做到支持v3s从spi-nand启动，我们有必要安装比较新一点的uboot代码。而之前的uboot都是从v3s官网下载，这个有点不一样的。新uboot的下载地址如下，我们选择了2022.04这个版本，

```
http://sources.buildroot.net/uboot/
```

在做一些uboot修改之前，我们先按照之前的方法编译一下uboot，也看看中间编译有没有什么问题，

```
make ARCH=arm LicheePi_Zero_defconfig
make ARCH=arm menuconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
```

在整个编译的过程当中，因为uboot的版本比较新，所以可能会遇到一些编译的问题。目前来说，涉及到的主要还是python库的一些安装，遇到问题之后，安装一下要求的库即可，

```
sudo apt-get install python3-distutils
sudo apt-get install libpython3-dev
```

有了最初的编译，即生成了最初的u-boot-sunxi-with-spl.bin，就可以开始调整和修改uboot内容了。首先拷贝spl_spinand_sunxi.c到arch/arm/mach-sunxi目录下，文件的地址来源如下，

```
https://github.com/bamkrs/openwrt/blob/dolphinpi-
spinand/package/boot/uboot-sunxi/src/arch/arm/mach-
sunxi/spl_spinand_sunxi.c
```

代码有两处需要修改，第一，添加头文件；第二，修改函数调用参数，

```
// add by feixiaoxing
#include <image.h>
#include <linux/delay.h>


ret = spl_parse_image_header(spl_image, bootdev, header); // modified
by feixiaoxing
```

接着，在arch/arm/mach-sunxi/Kconfig添加下面这些内容，

```
config SPL_SPINAND_SUNXI
    bool "Support for SPI Nand Flash on Allwinner SoCs in SPL"
    depends on MACH_SUN8I_V3S
    help
      Enable support for SPI Nand Flash. This option allows SPL to read
from
      sunxi SPI Nand Flash. It uses the same method as the boot ROM, so
does
      not need any extra configuration.
```

然后，在arch/arm/mach-sunxi/Makefile添加spl_spinand_sunxi.c的编译选项，我们的文件也需要放到CONIFIG_SPL_BUILD这个宏里面，

```
ifdef CONFIG_SPL_BUILD
obj-$(CONFIG_MACH_SUNIV)    += dram_suniv.o
obj-$(CONFIG_DRAM_SUN4I)    += dram_sun4i.o
```

```
obj-$(CONFIG_DRAM_SUN6I)    += dram_sun6i.o
obj-$(CONFIG_DRAM_SUN8I_A23)    += dram_sun8i_a23.o
obj-$(CONFIG_DRAM_SUN8I_A33)    += dram_sun8i_a33.o
obj-$(CONFIG_DRAM_SUN8I_A83T)   += dram_sun8i_a83t.o
obj-$(CONFIG_DRAM_SUN9I)    += dram_sun9i.o
obj-$(CONFIG_SPL_SPI_SUNXI) += spl_spi_sunxi.o
obj-$(CONFIG_SUNXI_DRAM_DW) += dram_sunxi_dw.o
obj-$(CONFIG_SUNXI_DRAM_DW) += dram_timings/
obj-$(CONFIG_DRAM_SUN50I_H6)    += dram_sun50i_h6.o
obj-$(CONFIG_DRAM_SUN50I_H6)    += dram_timings/
obj-$(CONFIG_DRAM_SUN50I_H616)  += dram_sun50i_h616.o
obj-$(CONFIG_DRAM_SUN50I_H616)  += dram_timings/
obj-$(CONFIG_SPL_SPINAND_SUNXI)    += spl_spinand_sunxi.o
endif
```

继续，在arch/arm/boot/dts/sun8i-v3s-licheepi-zero.dts文件当中添加spi0的内容，

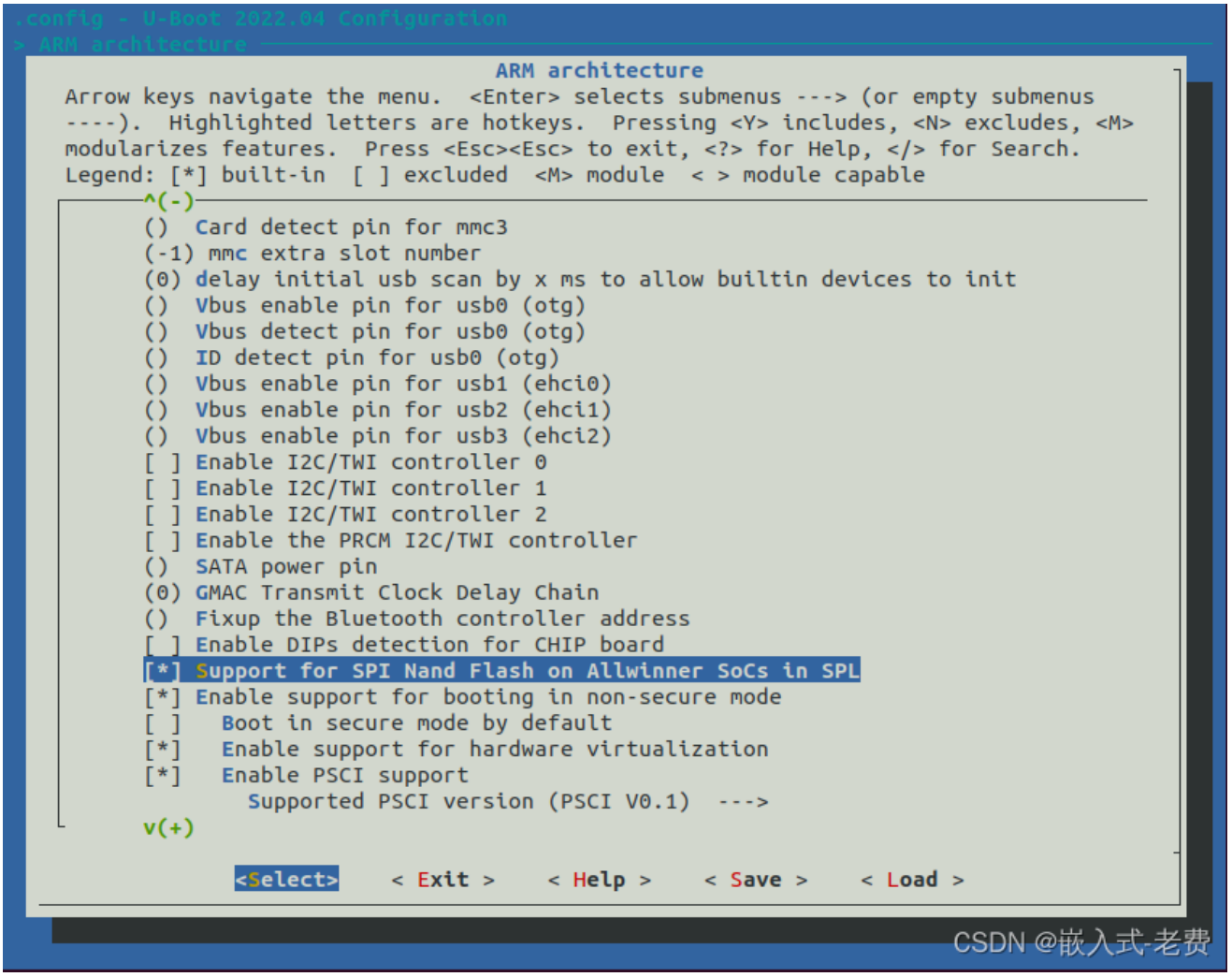
```
&spi0 {
    pinctrl-0 = <&spi0_pins>;
    pinctrl-names = "default";
    status = "okay";
    flash@0 {
        #address-cells = <1>;
        #size-cells = <1>;
        compatible = "spi-nand";
        reg = <0>;
        spi-max-frequency = <50000000>;
    };
};
```

还没有结束，继续在include/configs/sun8i.h添加这些内容，注意在#include <configs/sunxi-common.h>之前添加，

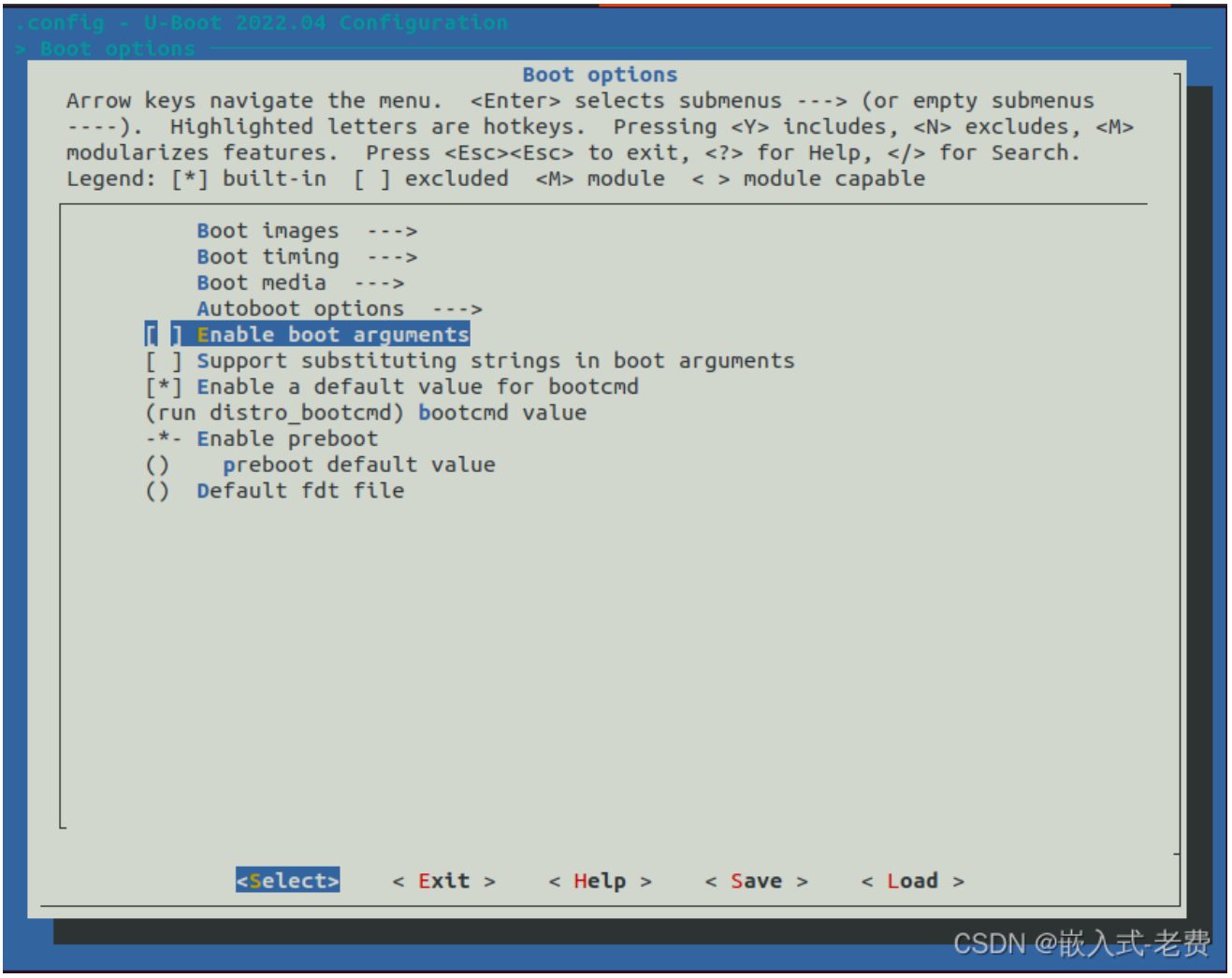
```
#ifdef CONFIG_BOOTCOMMAND
#undef CONFIG_BOOTCOMMAND
#endif

#ifdef COFNIG_BOOTARGS
#undef CONFIG_BOOTARGS
```

```
#endif


#define CONFIG_BOOTCOMMAND  "mtd read spi-nand0 0x41800000 0x100000
0x20000; "  \
                            "mtd read spi-nand0 0x41000000 0x120000
0x500000; " \
                            "bootz 0x41000000 - 0x41800000"

#define CONFIG_BOOTARGS    "console=ttyS0,115200 earlyprintk panic=5
rootwait " \

"mtdparts=spi0.0:1M(uboot)ro,128k(dtb)ro,5M(kernel)ro,-(rootfs)
ubi.mtd=3 root=ubi0:rootfs rw rootfstype=ubifs"
```

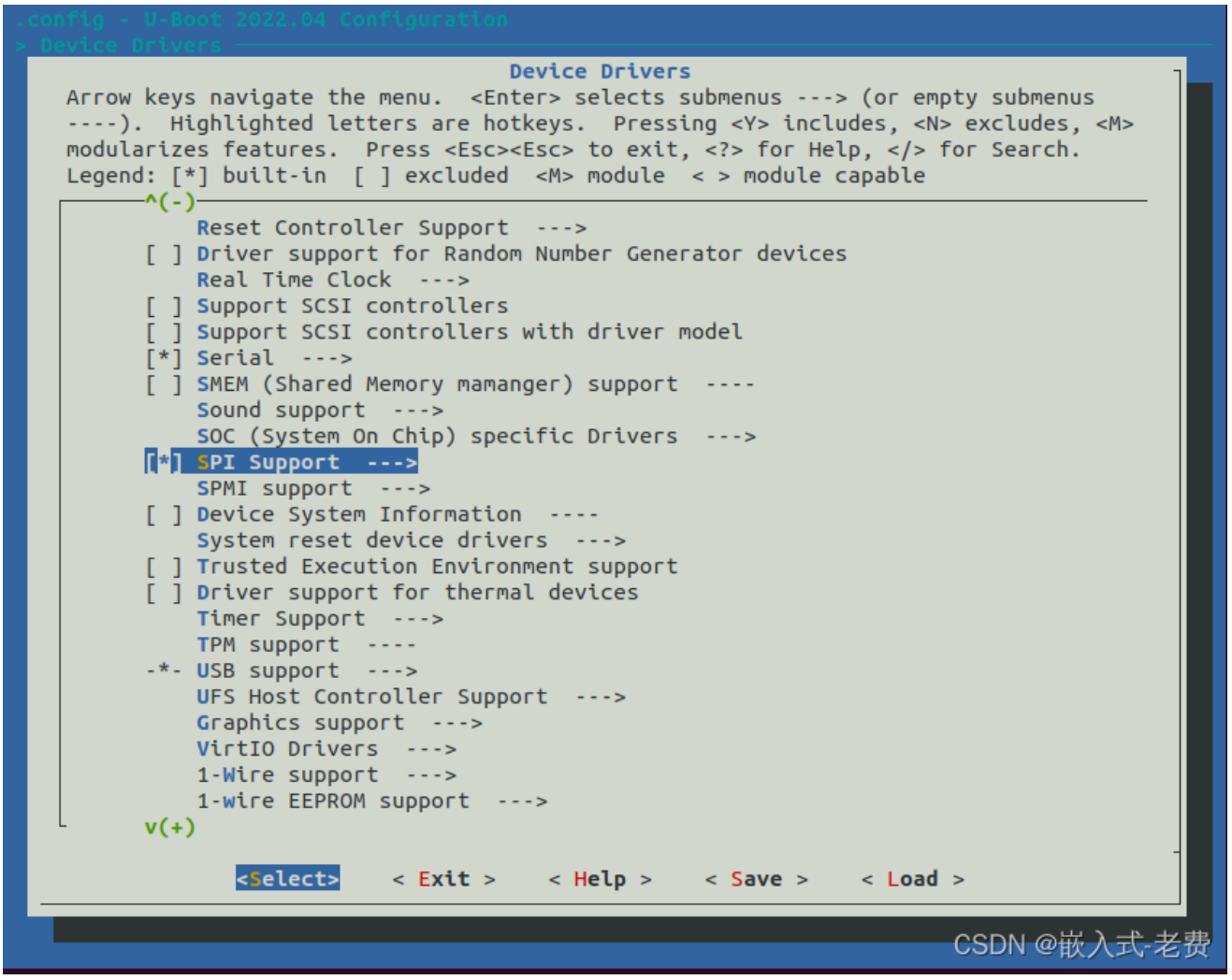
至此，uboot代码部分的修改已经结束了，下面就开始menuconfig的设置。menuconfig也大约有六个地方需要进行配置和确认，第一，ARM architecture勾选Support for SPI Nand Flash on Allwinner SoCs in SPL，这部分就是刚刚Kconfig添加的内容，



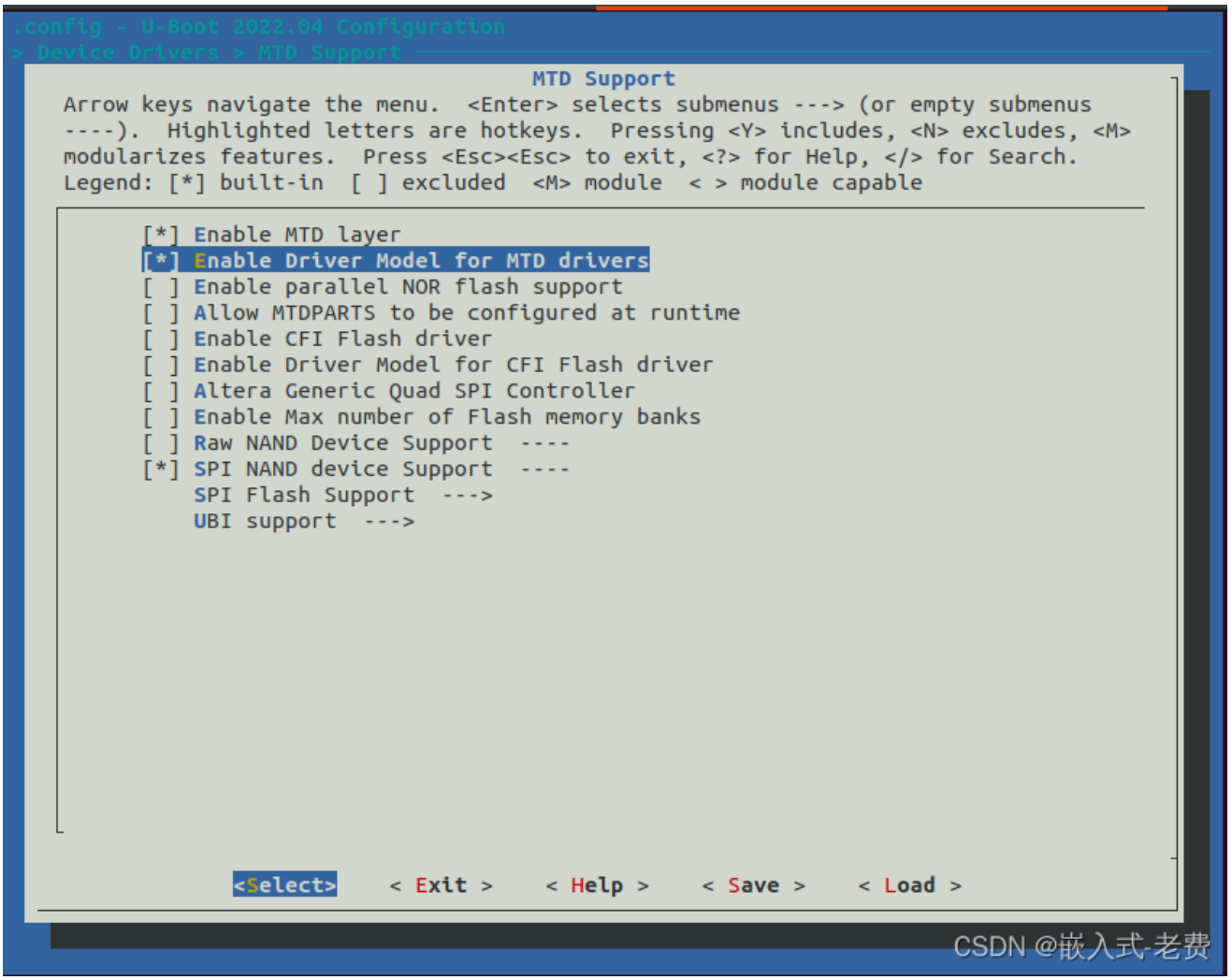
第二，Boot options取消掉Enable boot arguments，



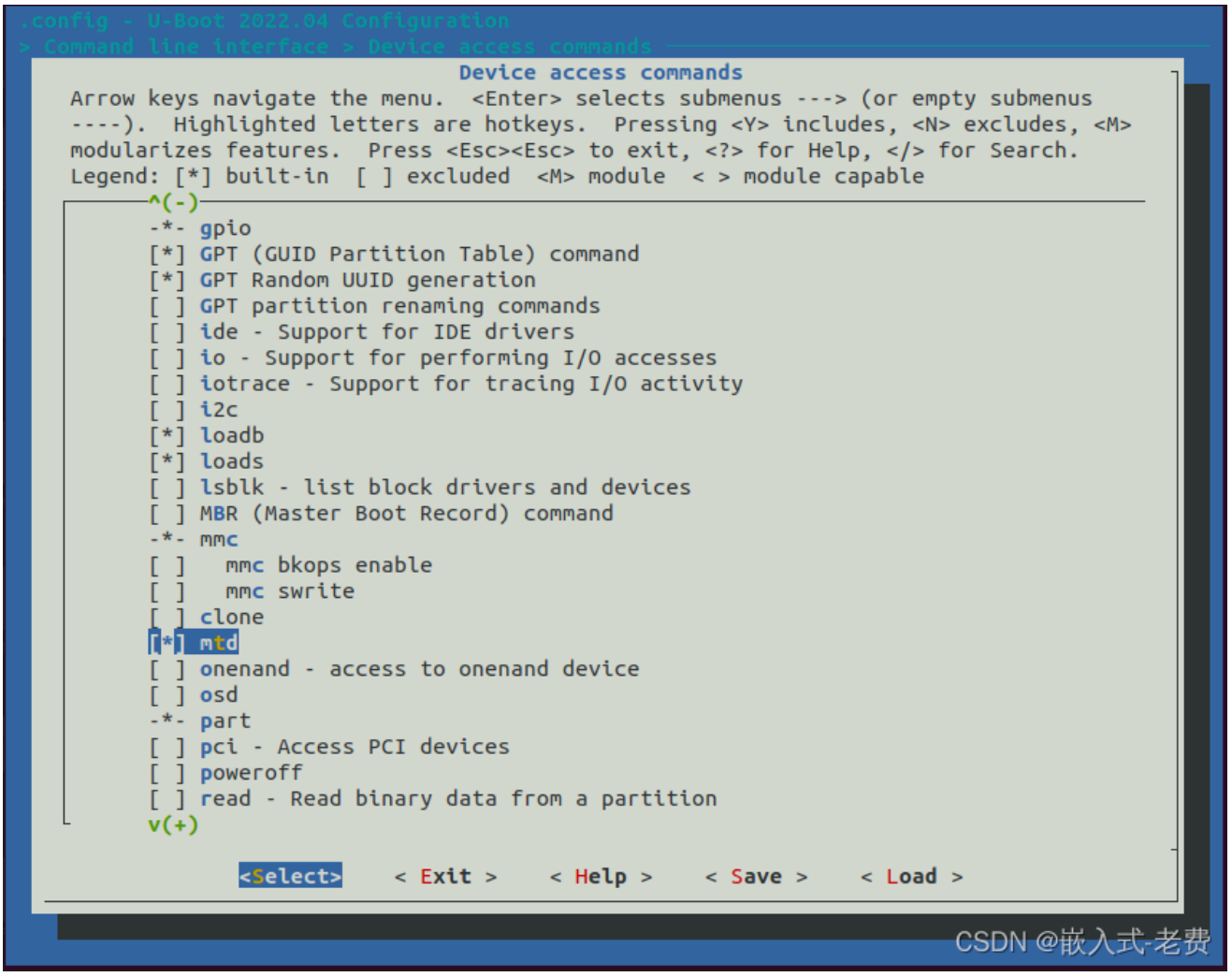
第三，打开Device Drivers下面的SPI Support，



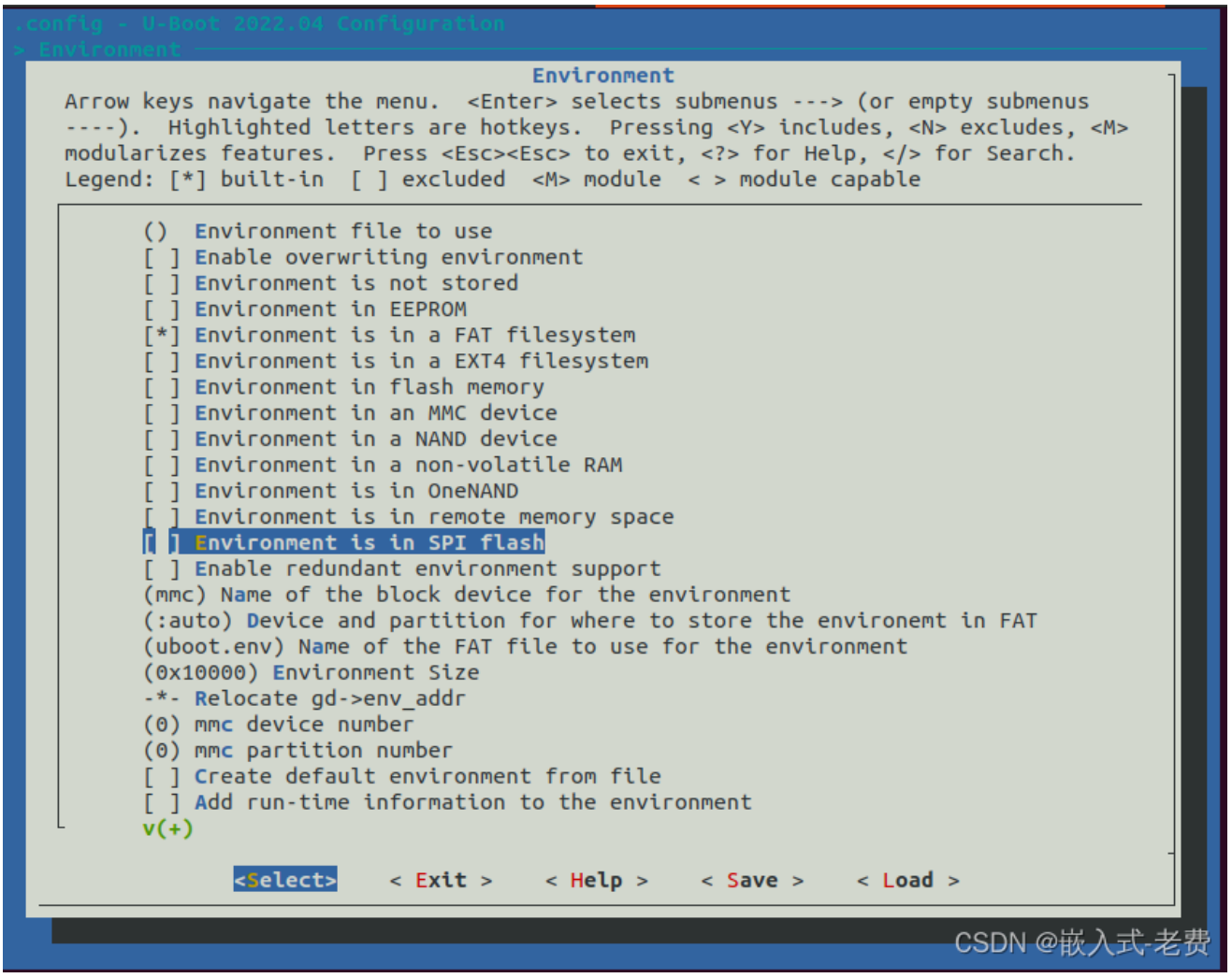
第四，在Deivce Drivers → MTD Support中勾选Enable MTD layer Enable Driver Model for MTD drivers和SPI NAND device Support，其中SPI Nand device Support这个选项一定是上面SPI Support被打开才会被看到的，



第五，在Command line interface→Device access commands勾选mtd，这样后续生成的uboot中才会有mtd这些命令，类似于为了处理spi-nor添加的sf命令，



第六，Environment取消勾选Environment is in SPI flash，同样这个选项也是打开SPI Support之后才会被看到的，



至此，我们对uboot的全部修改就已经结束了。下面要做的，就是利用make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-命令编译生成新的u-boot-sunxi-with-spl.bin文件。

3、利用sh脚本修改u-boot-sunxi-with-spl.bin文件

此时，u-boot-sunxi-with-spl.bin还不能被直接烧入到spi-nand。 **因为v3s默认从spi-nand加载page内容的时候，是1个page加载1k byte。但是mx35l1ge4ab这颗nand比较特殊，它的1个page是2k byte。** 当然，这里面有一定的历史原因，主要是因为v3s开发较早，当时普遍一个page都是1k，总不能要求v3s支持当时还不存在的nandflash芯片吧，所以这就需要一个脚本对u-boot-sunxi-with-spl.bin改造一下。脚本的链接地址如下，

```
https://github.com/bamkrs/openwrt/blob/dolphinpi-spinand/target/linux/sunxi/image/gen_sunxi_spinand_onlyboot_img.sh
```

使用方法也非常的简单，

```
./gen_sunxi_spinand_onlyboot_img.sh split.bin u-boot-sunxi-with-spl.bin
```

```
2048 128
```

其中split.bin代表目标文件，u-boot-sunxi-with-spl.bin表示刚刚编译好的uboot文件，2048代表page大小，单位是byte，而128则代表block大小，单位是k byte。

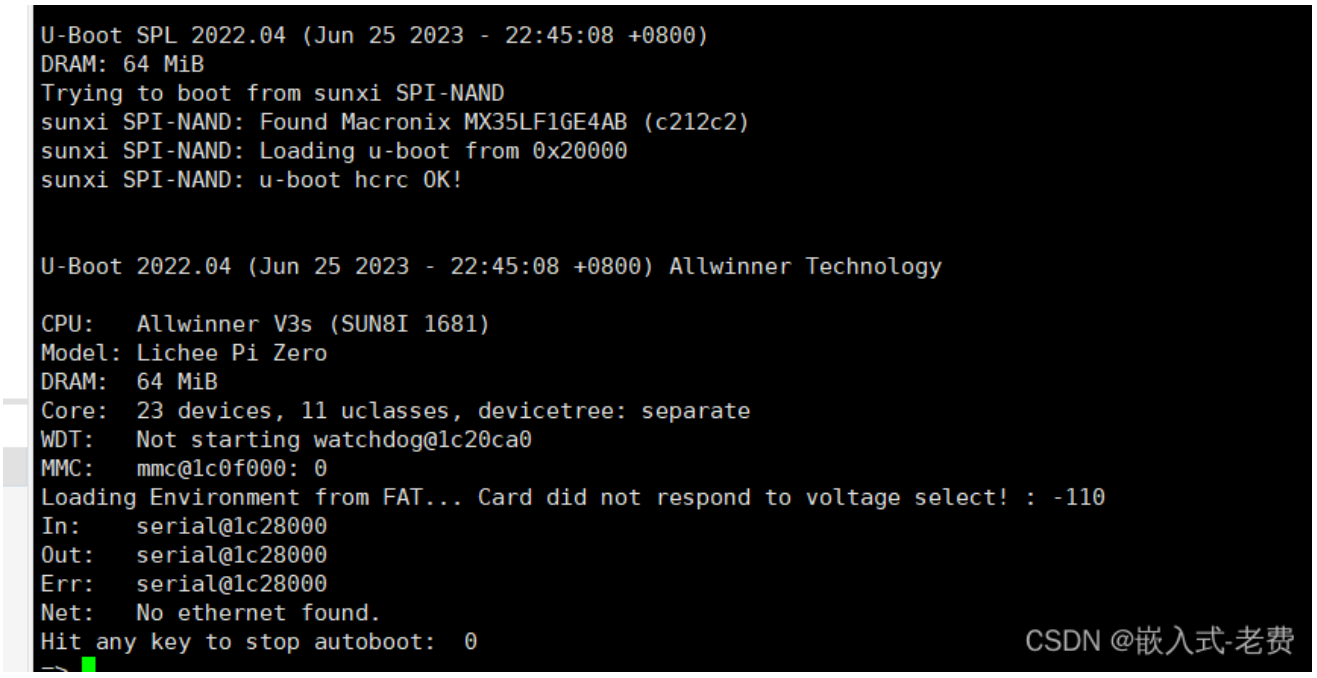
有兴趣的同学可以看一下，split前后的两个bin文件，用notepad++ 16进制插件对比下，有啥区别没有。

4、利用xfel烧入splt.bin文件

有了xfel工具和split.bin，这个时候就可以烧入uboot，看uboot是否可以正常起来了，

```
sudo ./xfel spinand erase 0x0 0x100000
sudo ./xfel spinand write 0x0 split.bin
```

烧入的位置位于nandflash最初的1M空间。没有其他问题的话，就可以看到uboot的正常打印信息了，



5、修改和烧入kernel文件

关于linux kernel的修改，主要有两个部分。第一个就是设备树的修改，第二个是menuconfig的配置。首先来看设备树，对应的文件是这个arch/arm/boot/dts/sun8i-v3s-licheepi-zero.dts，添加的内容如下所示，

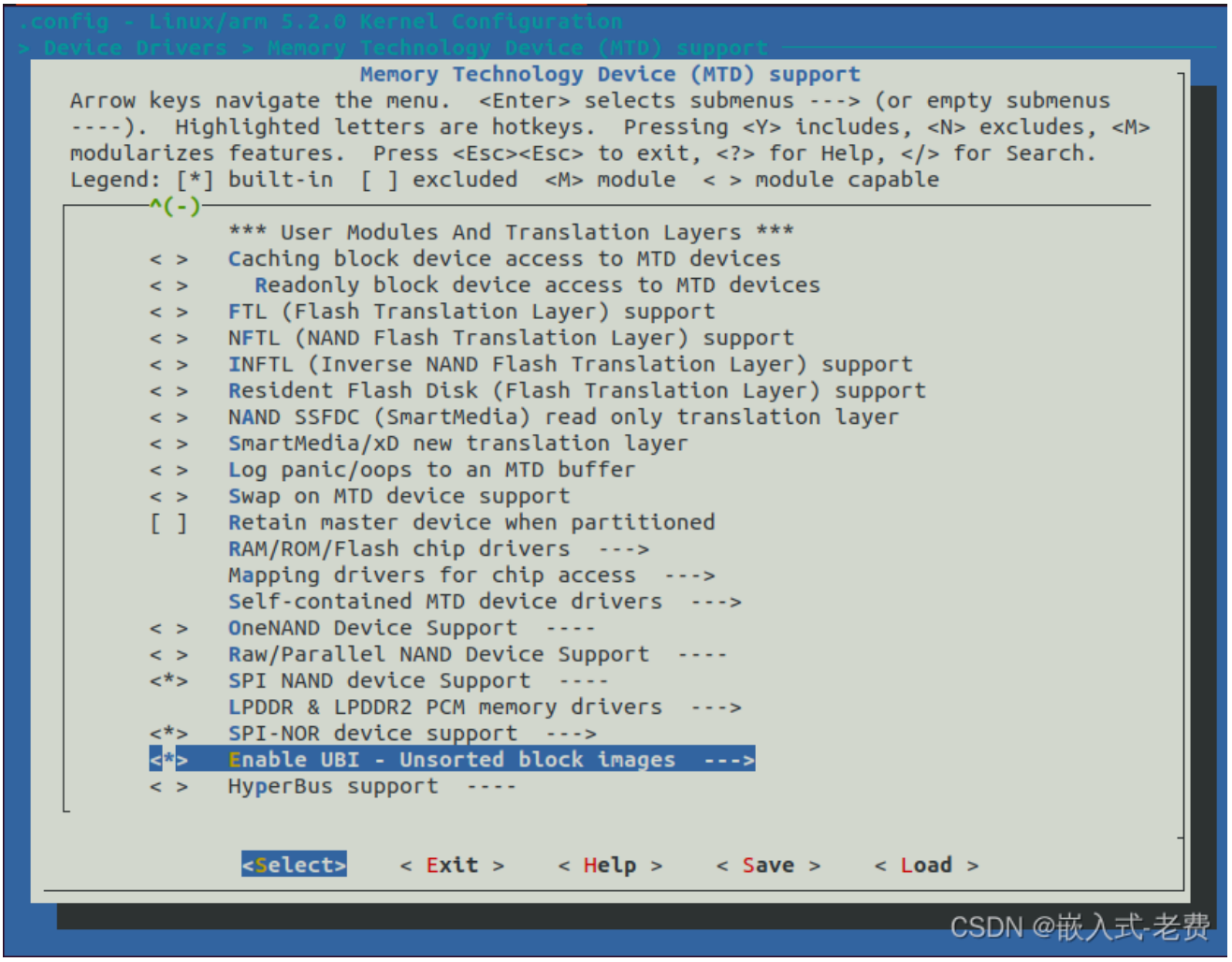
```
chosen {
        bootargs = "console=ttyS0,115200 earlyprintk panic=5 rootwait
mtdparts=spi0.0:1M(uboot)ro,128k(dtb)ro,5M(kernel)ro,-(rootfs)
```

```
ubi.mtd=3 root=ubi0:rootfs rw rootfstype=ubifs";
    };
```

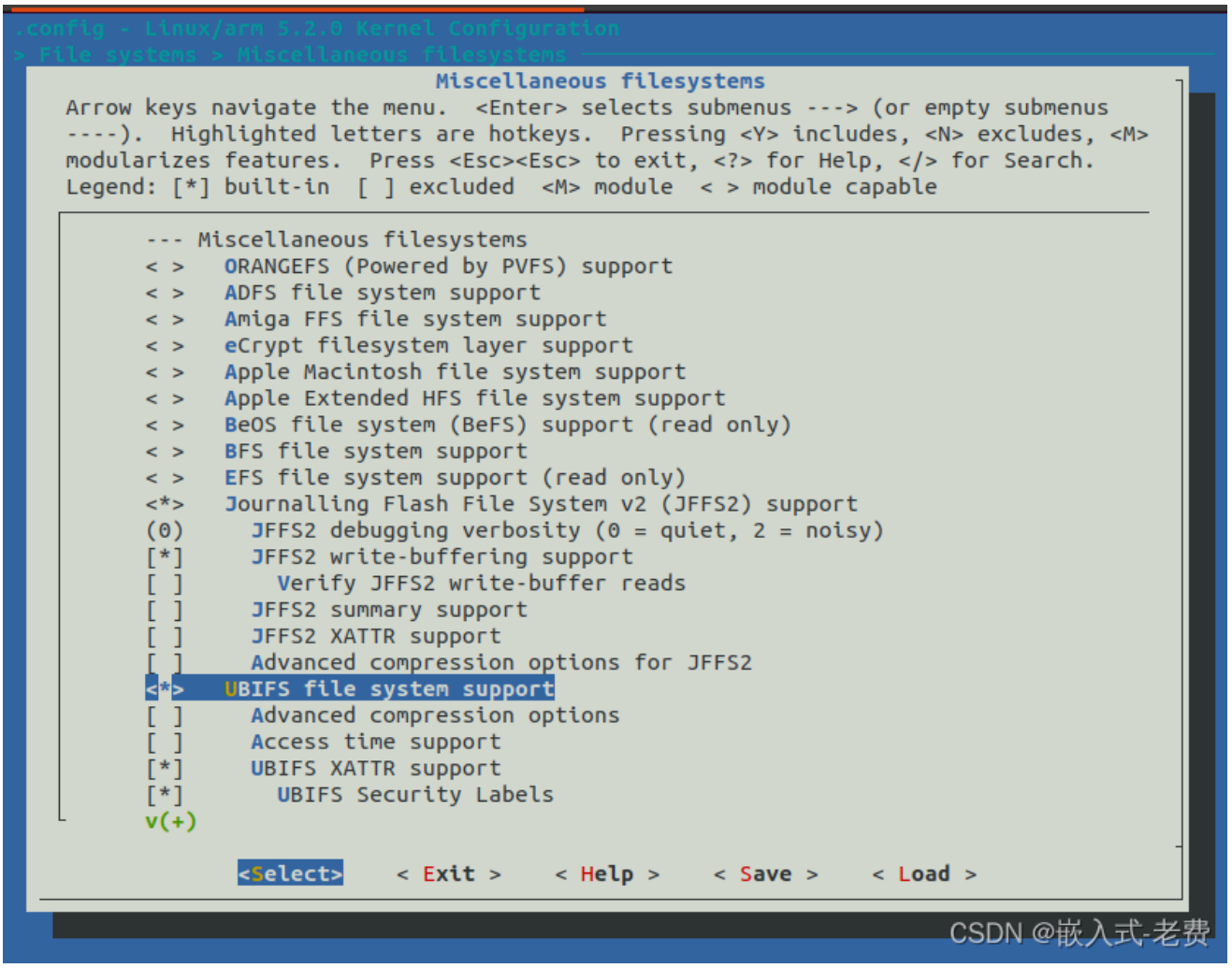
接着就是添加spi节点，这一点和uboot中的spi节点添加差不多，只是多了很多的partion分区内容，

```
spi0 {
    status ="okay";

    spi_nand: spi_nand@0 {
        #address-cells = <1>;
        #size-cells = <1>;
        compatible = "spi-nand";
        reg = <0>;
        spi-max-frequency = <50000000>;

        partition@0 {
            label = "uboot";
            reg = <0x0 0x100000>;
            read-only;
        };

        partition@100000 {
            label = "dtb";
            reg = <0x100000 0x20000>;
            read-only;
        };

        partition@120000 {
            label = "kernel";
            reg = <0x120000 0x500000>;
            read-only;
        };

        partition@620000 {
            label = "rootfs";
            reg = <0x620000 0x79E0000>;
        };
    };
};
```

完成好设备树的配置之后，就是对menuconfig进行配置了，也是有两个地方需要进行修改，首先在Memory Technology Device (MTD) support勾选SPI NAND device Support和Enable UBI - Unsorted block images，



接下来在File Systems的Miscellaneous filesystems中勾选UBIFS file system support，这一步需要在刚才的Enable UBI打开之后才能看到，



有了这些修改之后，就可以输入make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-生成新的dtb文件和zImage文件。和刚才uboot的烧入一样，这个时候也可以用xfel进行烧入处理，查看下，是不是uboot可以顺利跳转到kernel执行代码，

```
sudo ../xfel-master/xfel spinand erase 0x100000 0x20000
sudo ../xfel-master/xfel spinand write 0x100000 sun8i-v3s-licheepi-zero-with-480x272-lcd.dtb
sudo ../xfel-master/xfel spinand erase 0x120000 0x500000
sudo ../xfel-master/xfel spinand write 0x120000 zImage
```

当然，除了xfel之外，用uboot自身的mtd命令和y modem协议也是可以烧入的，

```
loady 0x41800000
loady 0x41000000

mtd erase spi-nand0 0x100000 0x20000
mtd erase spi-nand0 0x120000 0x500000

mtd write spi-nand0 0x41800000 0x100000 0x20000
```

```
mtd write spi-nand0 0x41000000 0x120000 0x500000
bootz 0x41000000 - 0x41800000
```

不出意外，我们可以看到uboot可以跳转到kernel执行，并且最终停在rootfs加载的地方，

```
[    1.653455] sun6i-rtc 1c20400.rtc: setting system clock to 1970-01-01T00:01:24 UTC (84)
[    1.661830] vcc3v0: disabling
[    1.664806] vcc3v3: disabling
[    1.667771] vcc5v0: disabling
[    1.670798] ALSA device list:
[    1.673765]   No soundcards found.
[    1.678607] VFS: Cannot open root device "ubi0:rootfs" or unknown-block(0,0): error -19
[    1.686717] Please append a correct "root=" boot option; here are the available partitions:
[    1.695144] Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
[    1.703405] CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.2.0-licheepi-zero #11
[    1.710528] Hardware name: Allwinner sun8i Family
[    1.715262] [<c010ede4>] (unwind_backtrace) from [<c010b774>] (show_stack+0x10/0x14)
[    1.723004] [<c010b774>] (show_stack) from [<c06e72b8>] (dump_stack+0x88/0x9c)
[    1.730224] [<c06e72b8>] (dump_stack) from [<c011dd64>] (panic+0x110/0x2fc)
[    1.737184] [<c011dd64>] (panic) from [<c0a0134c>] (mount_block_root+0x218/0x2f4)
[    1.744663] [<c0a0134c>] (mount_block_root) from [<c0a0154c>] (prepare_namespace+0x9c/0x194)
[    1.753094] [<c0a0154c>] (prepare_namespace) from [<c06fd9e8>] (kernel_init+0x8/0x10c)
[    1.761006] [<c06fd9e8>] (kernel_init) from [<c01010e8>] (ret_from_fork+0x14/0x2c)
[    1.768563] Exception stack(0xc3833fb0 to 0xc3833ff8)
[    1.773611] 3fa0:                                     00000000 00000000 00000000 00000000
[    1.781779] 3fc0: 00000000 00000000 00000000 00000000 00000000
```

```
00000000 00000000 00000000
[    1.789946] 3fe0: 00000000 00000000 00000000 00000000 00000013
00000000
[    1.796564] Rebooting in 5 seconds..
```

6、编译和烧入根文件系统

和之前的spi-nor使用jffs2不同，nandflash一般用ubifs文件系统比较多。 使用方法也比较简单，主要是两个步骤，涉及到两个工具，即mkfs.ubifs和ubinize。第一步，先用mkfs.ubifs生成 rootfs_ubifs.img 文件，其中-d target就是根文件目录的位置，

```
sudo mkfs.ubifs -x lzo -F -m 2048 -e 126976 -c 732 -o rootfs_ubifs.img
-d target
```

其次用ubinize生成最终的ubi.img，

```
sudo ubinize -o ubi.img -m 2048 -p 131072 -O 2048 -s 2048 ./ubinize.cfg
-v
```

命令行中提到的./ubinize.cfg如下所示，

```
[rootfs-volume]
mode=ubi
image=rootfs_ubifs.img
vol_id=0
vol_size=92946432
vol_type=dynamic
vol_name=rootfs
vol_flags=autoresize
vol_alignment=1
```

有了ubi.img之后，就可以用xfel工具烧入了。因为ubi.img实际空间差不多6M左右，所以按照16M的总大小来处理的话，可以依据16M-1M-128k-0×5M=0×9E0000来处理，

```
sudo ../xfel-master/xfel spinand erase 0x620000 0x9E0000
sudo ../xfel-master/xfel spinand write 0x620000 ubi.img
```

当然，这里的16M只是说烧入的大小，实际使用的rootfs空间取决于mkfs.ubifs和unibize中的空间设定，这是两个不同的概念。

7、整体测试

利用xfel分别烧入了split.bin、dtb、zImage和ubi.img之后，就可以开始整体的测试了。重启v3s开发板之后，不出意外的话，就可以看到这样的场景，基本说明我们的启动是成功的，

```
[    2.798351]   No soundcards found.
[    2.802313] ubi0: background thread "ubi_bgt0d" started, PID 92
[    2.810552] UBIFS (ubi0:0): Mounting in unauthenticated mode
[    2.819352] UBIFS (ubi0:0): background thread "ubifs_bgt0_0" started, PID 93
[    2.891484] UBIFS (ubi0:0): recovery needed
[    3.076239] UBIFS (ubi0:0): recovery completed
[    3.080833] UBIFS (ubi0:0): UBIFS: mounted UBI device 0, volume 0, name "rootfs"
[    3.088311] UBIFS (ubi0:0): LEB size: 126976 bytes (124 KiB), min./max. I/O unit sizes: 2048 bytes/2048 bytes
[    3.098236] UBIFS (ubi0:0): FS size: 91549696 bytes (87 MiB, 721 LEBs), journal size 9023488 bytes (8 MiB, 72 LEBs)
[    3.108679] UBIFS (ubi0:0): reserved for root: 0 bytes (0 KiB)
[    3.114521] UBIFS (ubi0:0): media format: w4/r0 (latest is w5/r0), UUID 9C65E13B-34CA-4ECA-90F2-378C0F4EE003, small LPT model
[    3.128016] VFS: Mounted root (ubifs filesystem) on device 0:13.
[    3.138083] devtmpfs: mounted
[    3.142331] Freeing unused kernel memory: 1024K
[    3.147124] Run /sbin/init as init process
Starting logging: OK
Initializing random number generator... done.
Starting network: OK

Welcome to Buildroot
buildroot login: root
#
#
```

CSDN @嵌入式-老费

最后，其实再看一下实际可以使用的rootfs空间大小，

```
# df
Filesystem           1K-blocks      Used Available Use% Mounted on
ubi0:rootfs              82000      2892     79108   4% /
devtmpfs                 26484         0     26484   0% /dev
tmpfs                    26996         0     26996   0% /dev/shm
tmpfs                    26996        24     26972   0% /tmp
tmpfs                    26996        16     26980   0% /run
#
```

CSDN @嵌入式-老费

这里看到空间大小有82M，目前只使用了2.8M，还有差不多80M，实际空间还是非常大的。和spi-nor相比较大的话，可用空间差不多大了3倍，毕竟当时spi-nor只有差不多20M的空间可以供外面使用。

最后给出最具有参考意义的两个链接地址，感谢网友的帮助，

1） https://whycan.com/t_8184.html

2） https://whycan.com/t_9244.html#p86894